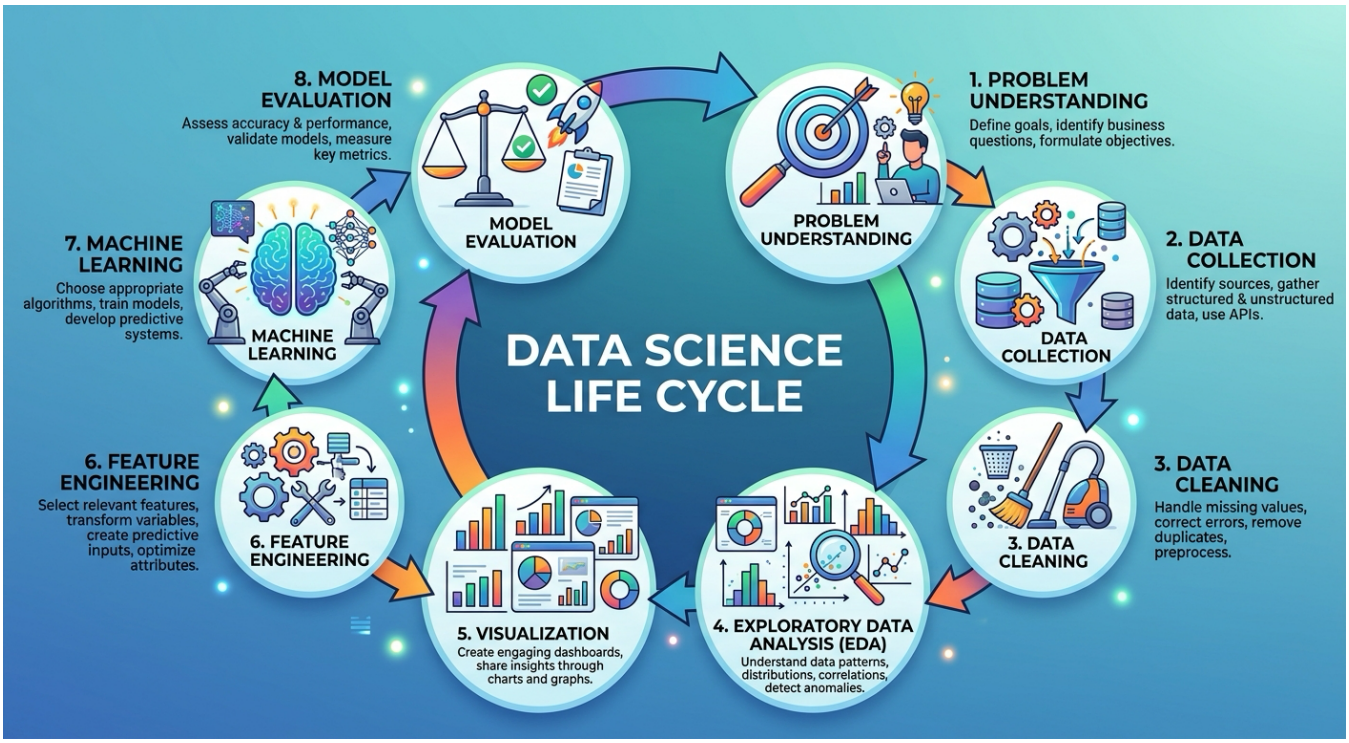
CSC 251 Lecture 17 - Machine Learning (Part 1)

Recall the Data Science Life Cycle:



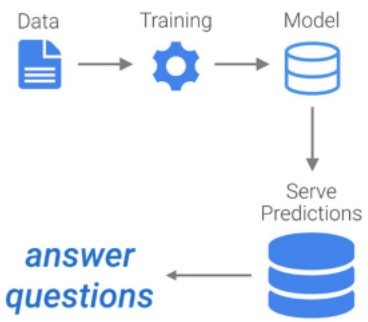
learning – "To get knowledge of something by study, experience, or being taught; to become aware by information or from observation; to commit to memory; to be informed of or to ascertain; to receive instruction." -New Oxford American Dictionary

For computers?

"Things learn when they change their behavior in a way that makes them perform better in the future."

What is machine learning?

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T, as measured by P, improves with experience E." -Tom Mitchell



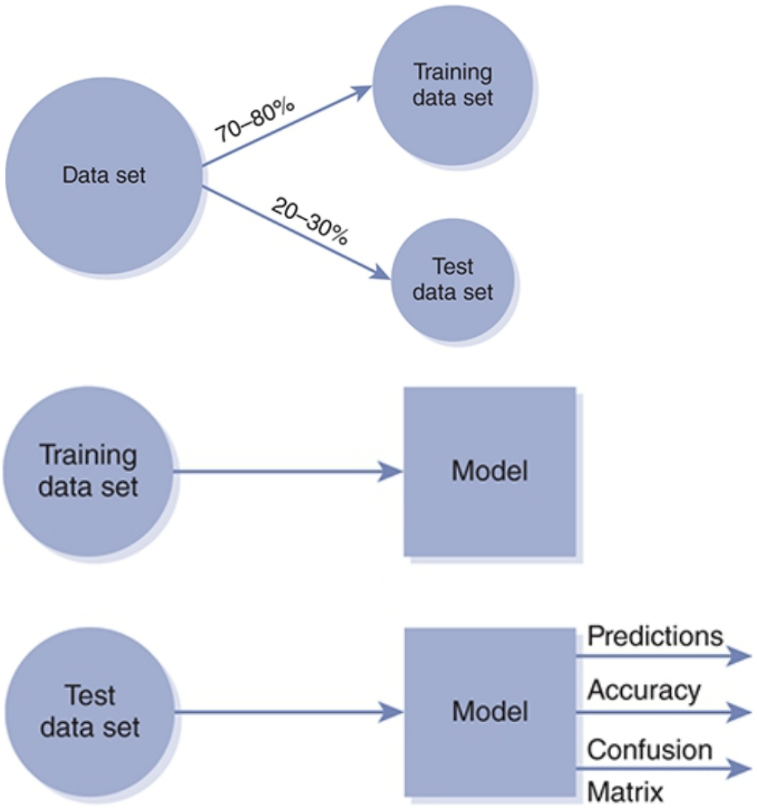
A subfield of Artificial Intelligence?

or

A means of building models from data in which finding optimal parameters results in improved performance?

A good machine learning system requires:

- Data preparation – clean, wrangle, split into a training set and a test set

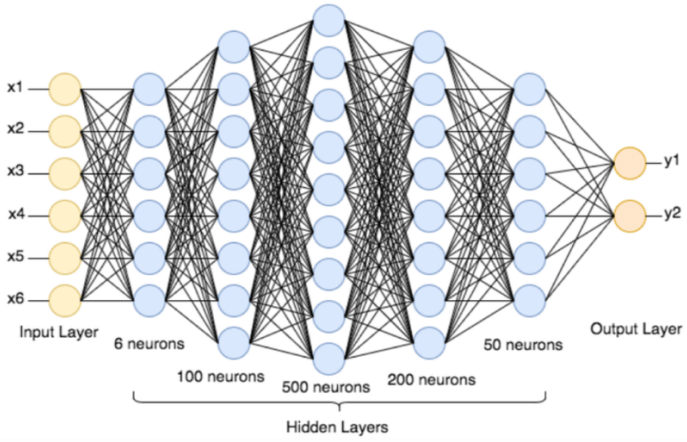


From: Data Mining and Analytics by Kris Jamsa

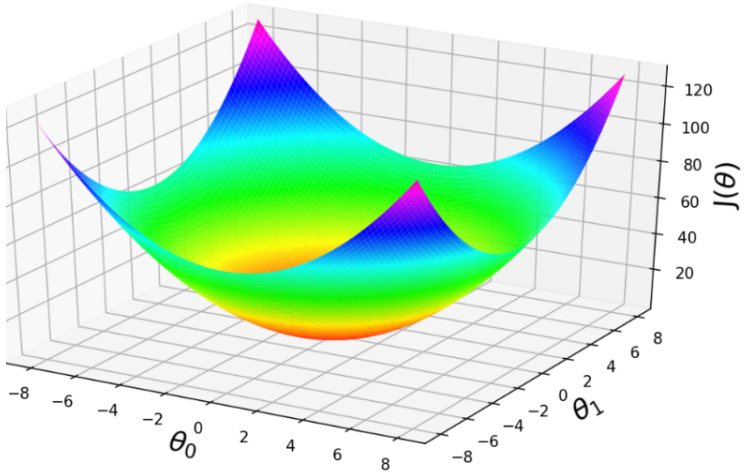
- Algorithms
  - Supervised learning – The labels on training examples are known, looks for patterns in the training set to make predictions about unlabeled testing data, classification algorithm used to determine the correct category for a data item.

  - Unsupervised learning – No labeled data or training set, looks for patterns in the data by discovery, clustering and data association algorithms used.

  - Deep learning – a structured process involving neural networks that leverages layers of machine learning in which the output of one layer becomes the input for the next.



- Automation and iterative processes
    - Regression analysis versus gradient decent – both can be processes



- Scalability
- Modeling

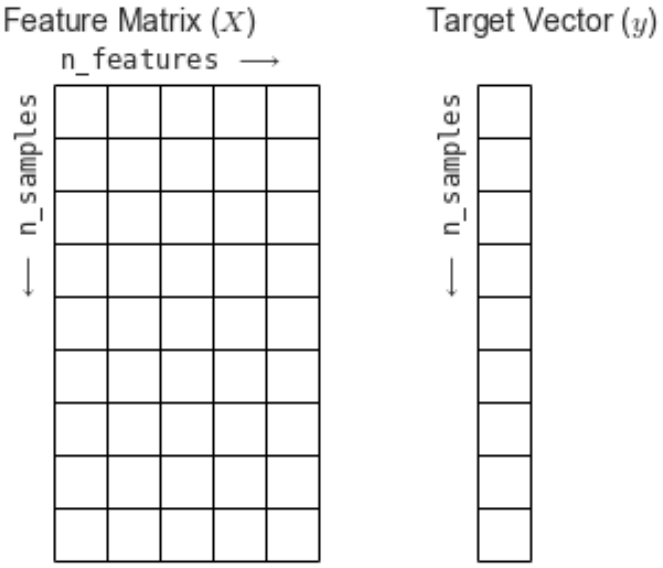
<u>Key Terminology:</u>

feature – one column or independent variable in your input set
feature engineering – converting data into values to build your feature matrix

label – the target or dependent variable, the ground truth that your output is compared against

X is the feature or predictor
Why is X uppercase? It could have multiple feature vectors, making it a matrix.

y is the label or response
Why y is lowercase?



Python Libraries for Machine Learning

1) Scikit learn (sklearn) – classical ML algorithms, data-mining, data analysis
2) TensorFlow – tensor computations, deep learning, neural networks
3) PyTorch – based upon C ML library Torch, computer vision, NLP
4) Theano – optimization utilizing CPU and GPU
5) Keras – fast prototyping, neural network design, runs on CPU and GPU
5) NumPy – large multi-dimensional array and matrix processing
6) SciPy – optimization, linear algebra, integration, stats, and image manipulation
7) pandas – data preparation and analysis
8) matplotlib – data visualization

Process of Machine Learning

Generally, we will complete the following steps:

1) Choose a class of model and import the appropriate class from package.

2) Choose model parameters and instantiate the class.

3) Arrange data into a features matrix and target vector

4) Fit the model to your data by calling the fit methods of the model instance

5) Apply the model to new data
    a) Predict labels for supervised learning through regression or classification
    b) Transform or infer properties for unsupervised learning

6) Evaluate the model and adjust the hyperparameters

Supervised Learning Classifiers

Classification – given a set of data points and their corresponding labels, "learn" how they are classified so that a new point can be put in the correct class.

A few common classification algorithms:

- Linear regression
- Logistic regression
- kNN (k-nearest neighbors)
- SVM (Support Vector Machine)
- Naïve Bays
- Random Forest
- K-means Clustering
- Decision Tree
- Gradient Descent

How do they differ?

- Performance
- Memory use
- Data set size
- Linearity
- Initial parameters

SciKit-learn

A widely-used Python package with general-purpose machine learning tools for both supervised and unsupervised methods. Utilizes `numpy` arrays so df data must be reshaped.

Two schools of thought:
Statistics (`statsmodels`) versus computer science/machine learning (`scikit-learn`)

To create a linear regression model, create a linear regression object, then perform a fit:

```
from sklearn import linear_model
lr = linear_model.LinearRegression()
predicted = lr.fit(X=tips['total_bill'], y =tips['tip'])
```

Common error:

```
Reshape your data either using array.reshape(-1, 1) if your data has a single feature or
array.reshape(1, -1) if it contains a single sample.
```

To correct, this we can reshape the data. With a simple `numpy` array:

```
arr = np.arange(8)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

```
arr.reshape(4,2)
```

```
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7]])
```

Where -1 can be used to avoid specifying the dimension:

```
arr.reshape(-1,1)
```

```
array([[0],
       [1],
       [2],
       [3],
       [4],
       [5],
       [6],
       [7]])
```

or

```
arr.reshape(1, -1)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

To transform a df column such as `tips['total_bill'].values`, perform simple linear regression and obtain the regression values:

```
predicted = lr.fit(X=tips['total_bill'].values.reshape(-1,1), y =tips['tip'])

predicted.intercept_
0.920269613554674

predicted.coef_
array([0.10502452])
```

For multiple regression:

```
pred2 = lr.fit(X=tips[['total_bill', 'size']], y = tips['tip'])
pred2.intercept_
0.6689447408125035

pred2.coef_
array([0.09271334, 0.19259779])
```

With categorical data, manually create dummy vars or use pandas get_dummies:

```
tips_dummy = pd.get_dummies(tips[['total_bill', 'size', 'smoker', 'day', 'time']])
```

	total_bill	size	smoker_No	smoker_Yes	day_Fri	day_Sat	day_Sun	day_Thur	time_Dinner	time_Lunch
0	16.99	2	True	False	False	False	True	False	True	False
1	10.34	3	True	False	False	False	True	False	True	False
2	21.01	3	True	False	False	False	True	False	True	False
3	23.68	2	True	False	False	False	True	False	True	False
4	24.59	4	True	False	False	False	True	False	True	False
...	...	...	...	...	...	...	...	...	...	...
239	29.03	3	True	False	False	True	False	False	True	False
240	27.18	2	False	True	False	True	False	False	True	False
241	22.67	2	False	True	False	True	False	False	True	False
242	17.82	2	True	False	False	True	False	False	True	False
243	18.78	2	True	False	False	False	False	True	True	False

Drop the first occurrence of each dummy variable then perform the regression:

```
tips_dummy_ref = pd.get_dummies(tips[['total_bill', 'size', 'smoker', 'day', 'time']],
drop_first=True)
```

	total_bill	size	smoker_Yes	day_Sat	day_Sun	day_Thur	time_Lunch
0	16.99	2	False	False	True	False	False
1	10.34	3	False	False	True	False	False
2	21.01	3	False	False	True	False	False
3	23.68	2	False	False	True	False	False
4	24.59	4	False	False	True	False	False

```
lr3 = linear_model.LinearRegression()
pred3 = lr3.fit(X=tips_dummy_ref, y = tips['tip'])

array([ 0.09427706,  0.17637418, -0.08647009, -0.12586354, -0.03257838,
       -0.16094587,  0.06814638])
```

The variable labels are dropped since they can't be held in `numpy` arrays. To append the names of the values:

```
values = np.append(pred3.intercept_, pred3.coef_)
names = np.append('intercept', tips_dummy_ref.columns)
results3 = pd.DataFrame(values, index=names, columns = ['coef'])
```

	coef
intercept	0.789582
total_bill	0.094277
size	0.176374
smoker_Yes	-0.086470
day_Sat	-0.125864
day_Sun	-0.032578
day_Thur	-0.160946
time_Lunch	0.068146

Compare with `statsmodels` analysis!

Example – Iris Flower Data

Edgar Anderson's iris data set used to quantify morphologic variation of 3 iris species, made famous by Ronald Fisher in 1936. Load from seaborn library:

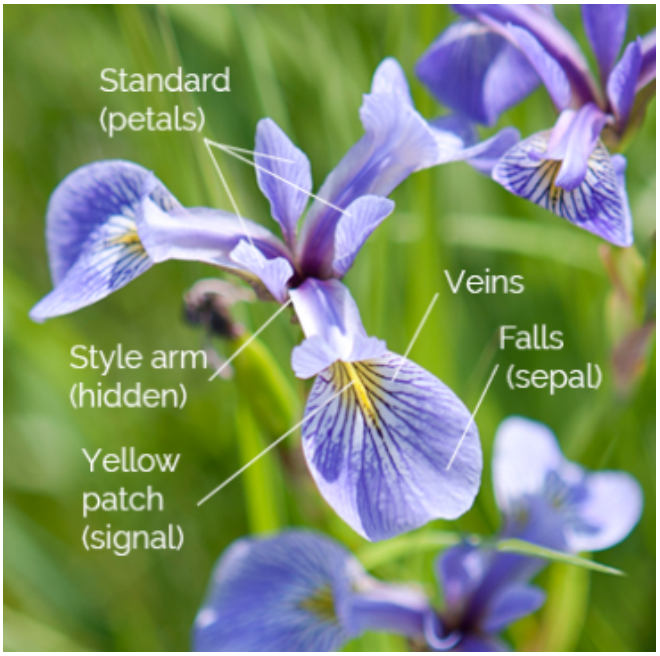
```
import seaborn as sns
iris = sns.load_dataset('iris')
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

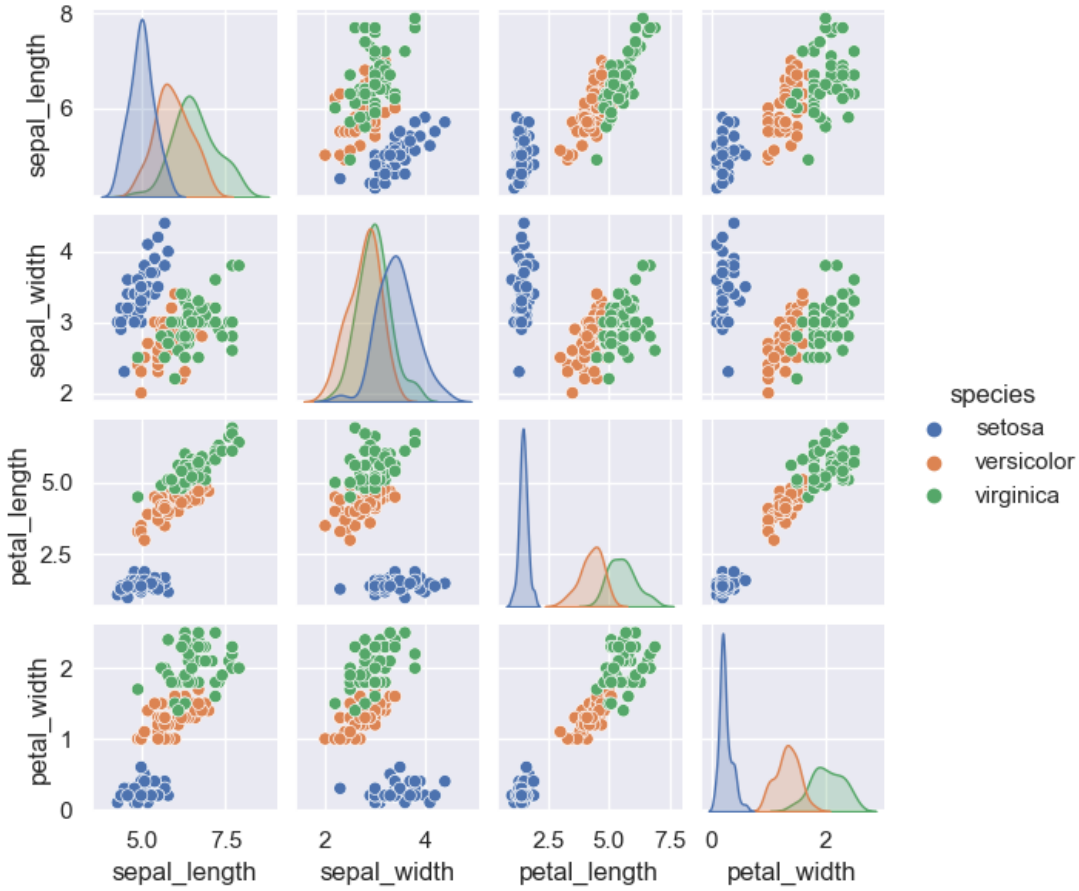
Each row is a sample obtained from a single observed flower.
Each column is a quantitative piece of information that describes each sample.

In case you're not familiar with iris anatomy:



Observations? Given _____, we would like to predict _____?

```
import seaborn as sns
sns.pairplot(iris, hue='species', height= 2)
```



Let's do a supervised learning model:

1) Choose a class of model and import the appropriate class from package.

Try the Naïve Bayes model, a group of extremely fast and simple classification algorithms that make general assumptions to create a general model. Bayes Theorem assumes independence among predictors. For the Gaussian Naïve Bayes classifier, it is assumed that the data for each label exhibits a simple Gaussian distribution. (skipping a lot of math here…)

```
from sklearn.naive_bayes import GaussianNB
```

2) Choose model parameters and instantiate the class.

No parameters are necessary for the Gaussian Naïve Bayes model.

```
model = GaussianNB()
```

3) Arrange data into a features matrix and target vector

Extract these arrays from the DataFrame:

a) The features matrix, X, with shape `[n_samples, n_features].`

```
X_iris = iris.drop('species', axis=1)
X_iris.shape
```

b) Our label or target array, y, is usually one-dimensional Series of length `n_samples`. This contains the quantity we want to predict from the data. What is the target for this data set?

```
y_iris = iris['species']
y_iris.shape
```

Split the data into training and testing sets:

```
from sklearn.model_selection import train_test_split
Xtrain, Xtest, ytrain, ytest = train_test_split(X_iris, y_iris, random_state=1)
```

4) Fit the model to your data by calling the fit method of the model instance

```
model.fit(Xtrain, ytrain)
```

5) Apply the model to new data - predict labels for supervised learning

```
y_model = model.predict(Xtest)
```

6) Evaluate the model and adjust the hyperparameters

How “good” was our model?

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{total number of predictions}}$$

```
from sklearn.metrics import accuracy_score
accuracy_score(ytest, y_model)
```

What did you get?